Introduction to Python for Finance

Practice Exercises — Functions

Instructions: define each function exactly as described, then call it at least once with your own test values to check the output. Unless the exercise says "print", your function must return the result, not display it. Read every Tip box before asking for help — it usually answers the most common mistake for that exercise.

Section 1 — Defining and Calling Functions

Goal: write your first functions and understand the difference between defining and calling them.

Exercise 1 — Your First Function

Define a function called welcome_message() that takes no parameter and prints: Welcome to the python Lab.
Then call it three times.

> **Tip:** Defining a function with def only stores it — nothing happens until you call it by writing welcome_message() on its own line.

Exercise 2 — A Function with One Parameter

Define a function greet_client(name) that prints a greeting including the client's name.
Call it with two different names.

> **Tip:** The name inside the parentheses is a parameter: a placeholder filled with whatever value you pass when you call the function.

Exercise 3 — print vs return

Define show_double(x) that prints x * 2.
Define get_double(x) that returns x * 2.
Call both with 50. Then try: result = show_double(50) and print(result). Explain in one comment line what you observe.

> **Tip:** A function without return gives back None. Use print() to display something to the user, and return to hand a value back to the rest of your program.

Section 2 — Return Values and Financial Calculations

Goal: turn standard finance formulas into reusable functions that return numbers.

Exercise 4 — Simple Interest

Define simple_interest(principal, rate, years) that returns principal * rate * years.
Call it with 1000, 0.05 and 3, and print the result.

> **Tip:** rate here is a decimal (0.05 means 5%). Be consistent across all your functions so you never mix 5 and 0.05 by mistake.

Exercise 5 — Compound Interest

Define compound_value(principal, rate, years) that returns the future value: principal * (1 + rate) ** years.
Print the value of a $2000 investment at 4% over 10 years.

> **Tip:** In Python, ** is the power operator — do not use ^, which means something completely different.

Exercise 6 — Percentage Return

Define pct_return(buy_price, sell_price) that returns the return as a percentage: (sell - buy) / buy * 100.
Test it with a profitable trade and a losing trade.

> **Tip:** Dividing by buy_price means the function breaks if buy_price is 0 — you will handle that case in a later exercise.

Exercise 7 — Convert a Currency

Define convert(amount, exchange_rate) that returns amount * exchange_rate.
Use it to convert 500 EUR into USD, then print the result rounded to 2 decimals using round().

> **Tip:** round(value, 2) returns a rounded number you can still use in calculations — keep the rounding for display, not for intermediate steps.

Exercise 8 — Average of a List

Define average(values) that takes a list of numbers and returns their mean.
Use a for loop and an accumulator inside the function (do not use the built-in sum()).
Test it with a list of daily closing prices.

> **Tip:** Remember that len(values) gives the number of items — divide the accumulated total by it at the end, after the loop, not inside it.

Section 3 — Default Values and Multiple Results

Goal: make functions more flexible with default arguments and return more than one value.

Exercise 9 — Default Interest Rate

Define future_value(principal, years, rate=0.03) with a default rate of 3%.
Call it once without the rate argument, and once with rate=0.07.

> **Tip:** Parameters with a default value must always come after the ones without a default, otherwise Python raises a SyntaxError.

Exercise 10 — Transaction Fee with a Default

Define net_amount(amount, fee_rate=0.01) that returns the amount after subtracting the fee.

Print the net amount for a 10,000 transfer with the default fee, then with a 2.5% fee.

> **Tip:** You can pass arguments by name when calling: net_amount(10000, fee_rate=0.025). This is clearer than relying on position.

Exercise 11 — Return Two Values at Once

Define min_max(prices) that returns both the lowest and the highest price of a list.

Call it and store the two results in two variables on one line: low, high = min_max(prices)

> **Tip:** A function returns several values by separating them with a comma in the return statement — Python packs them into a tuple you can unpack on the receiving side.

Section 4 — Functions Combining Conditions and Loops

Goal: reuse if / for / while inside functions, the way real financial tools are built.

Exercise 12 — Loan Approval Function

Define loan_decision(credit_score) that returns the string "Approved" if the score is 650 or above, and "Declined" otherwise.

Print the decision for three different scores.

> **Tip:** Use return inside each branch. Once a return runs, the function stops immediately — no need for an else in some cases.

Exercise 13 — Count Profitable Trades

Define count_winners(pnl_list) that takes a list of profit and loss values and returns how many of them are strictly positive.

Test it with [50, -20, 30, -10, 0, 75].

> **Tip:** Initialise your counter to 0 before the loop, increment it inside the if, and return it after the loop finishes.

Exercise 14 — Years to Reach a Savings Goal

Define years_to_goal(principal, rate, goal) that uses a while loop to grow the principal at the given rate until it reaches or exceeds the goal.

The function returns the number of years needed.

Test it with 1000, 0.06 and 2000.

> **Tip:** Make sure the balance changes inside the loop, otherwise the condition never becomes False and your program runs forever.